

DuaLoc: Dual-Encoder Bug Localization with Bug-Report-Conditioned Attention and Contrastive Learning

Amany AlBatlaa ^{1,2,*}  and Mohammad Abdullah-Al-Wadud ¹ 

¹ Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia

² Department of Cybersecurity, College of Computer, Qassim University, Qassim 52571, Saudi Arabia

* Correspondence: a.batlaa@qu.edu.sa (A.A.); mwadud@ksu.edu.sa (M.A.-A.-W.)

Abstract

Bug localization is the task of automatically identifying the source files responsible for a reported defect. It is a critical step in software maintenance that accelerates defect resolution. Information retrieval (IR) methods are simple and effective at exploiting historical signals such as bug-fixing recency and frequency, but they struggle to bridge the lexical gap between natural-language bug reports and programming-language identifiers. Recent work increasingly leverages pre-trained language models (PLMs) for code to close this gap. However, current PLM-based approaches still rely on a single code encoder that ignores program structure and aggregates function-level signals into file-level representations via uniform pooling. We propose a dual-encoder bug localization (DuaLoc) framework that jointly addresses these limitations. DuaLoc combines two pre-trained language models: UniXcoder for the semantic understanding of source code and GraphCodeBERT for awareness of data-flow structure. Both encoders are fine-tuned with a contrastive objective that shapes the embedding space around the localization task. A bug-report-conditioned attention mechanism then aggregates function embeddings into query-dependent file representations. The resulting neural similarity scores are then fused with classical IR features in a learning-to-rank model. DuaLoc outperforms representative classical and PLM-based baselines across most evaluation settings on a widely used benchmark of six open-source Java projects.

Keywords: bug localization; contrastive learning; learning-to-rank; pre-trained language models; software maintenance

1. Introduction

Software maintenance is the most resource-intensive phase of the software development life cycle. It accounts for up to 70% of the total project effort, a large share of which is driven by bug-related activity [1]. In maintenance, the first substantial task a developer faces when a bug is reported is to identify which parts of the source code are responsible for it. Modern projects routinely span thousands of files and millions of lines of code, and only a small fraction of that code is involved in any given defect. Manually inspecting candidate files to find the fault is no longer practical. A ranked list of suspicious files can narrow the search, thereby saving developers considerable inspection effort. Taking a free-text bug report and returning the source files most likely to contain the defect is known as bug localization [2,3].

Static bug-localization techniques broadly fall into two families: information retrieval (IR) and deep learning (DL). IR-based methods cast the bug report as a query and rank

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Electronics* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

source files by textual similarity [4]. Later work adds structural, version-history, and learning-to-rank features that keep these classical pipelines competitive [5,6]. Their main limitation is a lexical gap when the vocabulary used in a bug report does not overlap with the identifiers used in the source code.

Deep learning was adopted to close this gap through a succession of models, including deep networks built on top of IR features [7], convolutional and recurrent architectures that learn directly from bug reports and code [8,9], and richer code representations based on control-flow graphs and word-embedding token encodings [10,11]. Recent approaches are based on pre-trained language models (PLMs) such as FLIM [12] and fine-tuning CodeBERT [13] at the function level, because PLMs accept only a fixed number of input tokens. An entire source file typically exceeds this limit and must be truncated, whereas an individual function usually fits within it. Decomposing a file into its functions therefore preserves complete semantic units and avoids the context-length ceiling that handicaps file-level encoding.

Despite this progress, three gaps in existing PLM-based pipelines stand out. Firstly, nearly all of them rely on a single encoder that captures lexical alignment between comments and code but ignores program structure. Newer models such as GraphCodeBERT [14] and UniXcoder [15] address complementary aspects of code understanding; to our knowledge, they have not been combined for bug localization. Secondly, when function-level scores are aggregated into a file score, prior work relies on max or mean pooling, which weights every function in a file equally. In practice, only a handful of functions are implicated by any given bug, so a uniform aggregation discards which functions actually matter for the report at hand. Third, although contrastive and ranking-based objectives have been explored individually, their interaction with a dual-encoder architecture and a query-conditioned aggregation step remains largely unstudied.

We propose DuaLoc (Dual-encoder Localization), a framework built around three design choices that directly target the limitations identified above. First, a dual-encoder backbone pair is UniXcoder for cross-modal semantics and GraphCodeBERT for data-flow awareness. The lexical and structural features are extracted by models pre-trained for that purpose. Second, a bug-report-conditioned attention mechanism aggregates function embeddings into a file representation that varies with the query. This allows the same file to be ranked differently against different bug reports. Third, both encoders are fine-tuned with an InfoNCE contrastive objective [16] using in-batch negatives, so the embedding space is shaped directly by the retrieval task. The resulting deep similarity scores are fused with classical IR features in a pointwise learning-to-rank model trained with hard-negative mining over the top files returned by surface lexical similarity [17].

We evaluate DuaLoc on the benchmark of Ye et al. [17], which spans six widely studied open-source Java projects: Eclipse Platform UI, JDT, BIRT, SWT, Tomcat, and AspectJ. We also compare it against four representative baselines: BugLocator [4], YBL [17], DreamLoc [18], and FLIM [12]. DuaLoc outperforms the baselines in the vast majority of project-metric combinations across the six projects and the three standard metrics (Top-K accuracy, MAP, and MRR). Two ablation studies further show that the IR, UniXcoder, and GraphCodeBERT signals are complementary rather than redundant and that both contrastive fine-tuning and bug-report-conditioned attention contribute materially to the final ranking.

The main contributions of this paper are the following:

1. A dual-encoder bug localization framework that pairs a semantically oriented PLM (UniXcoder) with a data-flow-aware PLM (GraphCodeBERT) and fuses their similarity scores with IR features in a learning-to-rank model.

2. A bug-report-conditioned attention mechanism that aggregates function embeddings into file-level representations.
3. A contrastive fine-tuning procedure based on InfoNCE with in-batch negatives, which shapes the embedding geometry directly around the localization objective.
4. An empirical evaluation on six open-source Java projects against baselines, supported by ablation studies that isolate the contribution of each feature group and each architectural component.

The rest of the paper is organized as follows. Section 2 reviews background on the task and the two pre-trained models we built on. Section 3 surveys related work. Section 4 presents the proposed method in detail. Section 5 describes the experimental design, and Section 6 reports and discusses the results. Section 7 discusses threats to validity, and Section 8 concludes.

2. Background

This section provides the background for DuaLoc: the bug localization task, the two pre-trained language models, UniXcoder and GraphCodeBERT, and the learning-to-rank formulation.

2.1. Bug Localization

In large-scale software systems, manually locating defects through inspection is both time-consuming and labor-intensive. Bug localization frames this task as an automated retrieval problem. The system returns a ranked list of source files most likely to contain the defect, based on the submitted bug report. The quality of this ranking determines how quickly developers can move from a reported bug to the responsible code, and it is therefore the central evaluation target of the field.

Existing techniques are conventionally categorized as dynamic or static [3]. Dynamic methods, such as spectrum-based fault localization [19], analyze runtime traces and rely on comprehensive test suites. Static methods instead use project artifacts such as source code, bug reports, and version control history. Within the static family, information retrieval-based techniques were the dominant early paradigm, treating localization as text matching between reports and code [4,5]. Deep learning approaches have since become the leading line of work by modeling semantic relationships that surface terms cannot capture [8,10]. DuaLoc is positioned at the intersection of these two strands, combining IR features with learned semantic and structural representations within a single learning-to-rank model.

2.2. UniXcoder

UniXcoder [15] is a Transformer-based pre-trained model for source code. It follows the same approach used by natural-language encoders such as BERT [20] and RoBERTa [21]. It has been jointly trained on code and natural language and maps the two modalities into a shared representation space rather than using separate encoders. It follows a BERT-style input format [20] in which a [CLS] token aggregates a sequence of subword tokens. A special prefix token then selects which of three operating modes the model uses at inference time: encoder-only, decoder-only, or encoder-decoder. UniXcoder is pre-trained with three language modeling objectives: masked language modeling for the encoder, unidirectional language modeling for the decoder, and a denoising objective for the encoder-decoder.

Two properties make UniXcoder well-suited to our framework. First, the encoder-only mode produces fixed-length representations for both bug reports and source-code functions, as required by the bi-encoder design of Section 4.2. Second, UniXcoder is pre-trained with a cross-modal generation task that aligns natural-language descriptions with their corresponding code. This directly matches the bug report to the code retrieval problem

at the core of our task. The cross-modal pre-training motivates our choice of UniXcoder as the semantic encoder of the dual-encoder pair.

2.3. GraphCodeBERT

GraphCodeBERT [14] is a Transformer-based [22] pre-trained model that injects the semantic structure of source code into its representations through data flow. For each function, it extracts a data flow graph whose nodes are variable occurrences and whose directed edges encode the source of each variable's value. It uses a graph-guided masked attention mechanism to incorporate this structure into the encoder. Attention is allowed in three cases: First, between code and comment tokens. Second, between variable nodes that are connected by a data-flow edge. Third, between a variable node and the code token from which it was identified. The model is pre-trained on the CodeSearchNet corpus with three objectives: masked language modeling, data-flow edge prediction, and node alignment between source code and data flow.

Two aspects of GraphCodeBERT motivate its role in our framework. First, its data-flow-aware representations capture dependency information that purely token-based encoders are not designed to expose. This makes it a natural complement to the semantic encoder of the dual-encoder pair introduced in Section 4.2. Second, because GraphCodeBERT is itself a Transformer encoder, it is architecturally compatible with UniXcoder and can be trained using the same training recipe.

2.4. Learning to Rank

The learning-to-rank approach focuses on the ordering of candidate items rather than on any individual prediction about them. The standard taxonomy distinguishes three families: Pointwise methods score each item independently and reduce ranking to a regression or classification task [6]. Pairwise methods instead learn from item pairs to determine their relative order [23,24], whereas listwise methods take the entire ranked list as the unit of supervision and optimize a list-level loss directly [25].

Bug localization fits this paradigm, in which a query is a bug report and the repository's source files are the candidate items. The features are textual overlap, semantic similarity, and historical bug-fixing patterns, which can be combined into a single scoring function rather than handled by separate heuristics. We follow this line and adopt a pointwise formulation, in which each (bug report, file) pair is scored independently. The feature vector defined in Section 4.4, comprising deep similarity and IR features, drives the final ranking score.

3. Related Work

This section surveys previous bug localization approaches, organized into information retrieval-based and deep learning-based categories.

3.1. Information Retrieval-Based Approaches

The first wave of automated bug-localization research framed the problem as a text-retrieval task between bug reports and source files. Zhou et al. [4] introduced BugLocator, which builds on a revised Vector Space Model (rVSM) that accounts for file length and additionally incorporates the textual similarity of previously fixed reports as a secondary ranking signal. Saha et al. [5] proposed BLUiR that moves beyond bag-of-words matching by treating source files as structured documents. Class names, method names, variable names, and comments are indexed as separate fields, and similarity is computed between each bug-report field and each code field. Wang and Lo [26] extended this multi-source view in AmaLgam, which fuses three sources of evidence, similar past reports, version history, and the structured-IR view of BLUiR into a single ranking. Youm et al. [27] further enriched

the BLIA retrieval pipeline by integrating source-code modification history, structural decomposition of source files, bug report text, and stack traces. Wen et al. [28] proposed Locus, which shifts the unit of retrieval from source files to change hunks. Each hunk is represented by a natural-language corpus of changed lines, context, and commit log alongside a code-entity corpus of package, class, and method names, with change-history information incorporated as a boosting signal at both the file and the inducing-change level. Gharibi et al. [29] designed a multi-component pipeline in which token-level matching between bug reports and source files is combined with stack-trace information and a multi-label classifier trained on previously fixed bug reports, and the component outputs are fused at ranking time.

A second wave reformulated bug localization as a supervised learning-to-rank (LTR) problem that combines heterogeneous evidence under learned feature weights. Ye et al. [30] introduced the first such formulation, defining a feature vector that encodes textual similarity, surface lexical similarity at the method level, API-document similarity, collaborative filtering over previously fixed similar reports, and bug-fix recency and frequency, with weights learned through SVM-rank. Ye et al. [17] later extended this model by incorporating additional structural IR features computed between bug report components and parsed code elements, such as class names, method names, variable names, and comments, as well as file dependency graph features derived from PageRank and HITS. Fejzer et al. [6] proposed an adaptive pointwise LTR algorithm that automatically tunes feature weights to project characteristics without a separate parameter-setting step. It uses statistical test-based feature weighting together with an improved training target that produces continuous relevance values, ensuring that every buggy file ranks above every non-buggy one. Despite their simplicity, scalability, and effective use of historical signals, IR-based approaches share a fundamental limitation. They rely primarily on lexical overlap between bug reports and source code and therefore struggle to bridge the lexical gap that arises when developers describe a bug using vocabulary that does not appear directly in the code.

3.2. Deep Learning-Based Approaches

The first line of work on deep-learning-based bug localization applied neural networks to capture semantic relationships beyond surface-level text matching. Lam et al. [7] integrated a deep neural network (DNN) with rVSM, where the DNN supplies the semantic alignment between bug reports and code while rVSM continues to handle direct lexical overlap. Huo et al. [8] later proposed NP-CNN, which extracts joint features from natural language and source code through convolution operations tailored to program structure, learning a unified representation while respecting the syntactic units of the code. Xiao et al. [31] proposed CNN-Forest, which combines convolutional neural networks with cascade forests to model the connection between bug reports and source files. The same authors later introduced DeepLoc [11], an enhanced CNN-based model that directly incorporates bug-fixing recency and frequency into the cost function. Liang et al. [32] proposed CAST, which combines a tree-based convolutional neural network with a customized abstract syntax tree to better capture lexically and semantically rich code. Huo et al. [10] formulated bug localization as a multi-instance learning problem in which each source file is represented as a bag of basic block instances extracted from its control flow graph. Yang and Lee [9] later proposed a CNN-LSTM hybrid that exploits topic-based similarity between past commits. This generation of models shares recurring constraints despite their gains over purely lexical retrieval. It includes a limited receptive field that struggles with long-range dependencies common in source code, as well as RNN components that are vulnerable to gradient vanishing. Moreover, they continue to rely on lexical features, leaving deeper program structure unexploited.

A second line of work shifted toward attention mechanisms and pre-trained language model-based work. Qi et al. [18] proposed DreamLoc, a deep relevance-matching framework that uses attention together with a wide and deep architecture to compute matching scores between bug reports and code snippets. Because it operates on fixed-length text segments that often cut across method boundaries, it disrupts the code's syntactic and semantic coherence. Liang et al. [12] proposed FLIM, which fine-tunes CodeBERT [13] at the function level and fuses the resulting semantic similarity scores with IR features inside a learning-to-rank model. It is designed to overcome the context length limit of transformer encoders when applied to long source files. These approaches share three limitations despite their advances: a single encoder that ignores program structure, uniform max or mean pooling that weights all functions equally, and a cross-entropy objective that does not shape the embedding space for retrieval. DuaLoc belongs to this family of PLM-based approaches but addresses these three limitations together through a dual-encoder design, a bug report-conditioned attention aggregation, and a contrastive fine-tuning objective.

The third line of work has begun applying large language models (LLMs) to bug localization. Asad et al. [33] proposed GenLoc, an LLM-based technique that combines semantic retrieval with code-exploration functions, allowing the model to iteratively analyze the codebase via embedding-based similarity and ReAct-guided reasoning. Shao and Yu [34] proposed LLmiRQ, which uses GPT-4 to construct queries for IR-based fault localization by categorizing bug reports into programming entities, stack traces, and pure natural language before feeding the result into a learning-to-rank model. An extended version, LLmiRQ+, further refines unsuccessful queries through a user-driven conversational loop of up to five iterations. Chakraborty et al. [35] proposed BLAZE, a cross-language and cross-project framework that segments source code at natural component boundaries through a dynamic-programming chunking algorithm. It then fine-tunes a GPT-based embedding model with hard examples mined directly in the embedding space, fusing lexical and deep retrievers through reciprocal rank fusion. While this line of work makes promising use of large language models, it shares persistent limitations. The first is a heavy reliance on closed-weight commercial models, whose non-deterministic outputs and provider-version changes make results difficult to reproduce. The second is the use of human-in-the-loop refinement loops, which shift a substantial portion of the localization effort back onto the developer. Finally, these approaches continue to rely solely on text-based embeddings, excluding program structure from the similarity computation.

4. The Proposed Method

Effective bug localization requires complementary forms of evidence: the semantic and structural correspondence between a bug report and the source code it implicates. It also requires information retrieval features to capture lexical overlap and bug-fixing patterns derived from project artifacts. As illustrated in Figure 1, the proposed framework integrates two pre-trained code-language models: UniXcoder [15], which provides cross-modal semantic alignment, and GraphCodeBERT [14], which encodes data-flow structure together with classical information retrieval features that capture lexical similarity, collaborative filtering, and bug-fixing recency and frequency. The two encoders are first adapted for the retrieval task via contrastive fine-tuning on (bug report, function) pairs. The fine-tuned encoders then perform as feature extractors. A bug-report-conditioned attention mechanism aggregates function-level embeddings into a query-dependent file-level representation. The resulting deep similarity scores are concatenated with the information retrieval features into a single vector. This vector is then passed to a pointwise linear ranker, producing the final ordering of candidate source files.

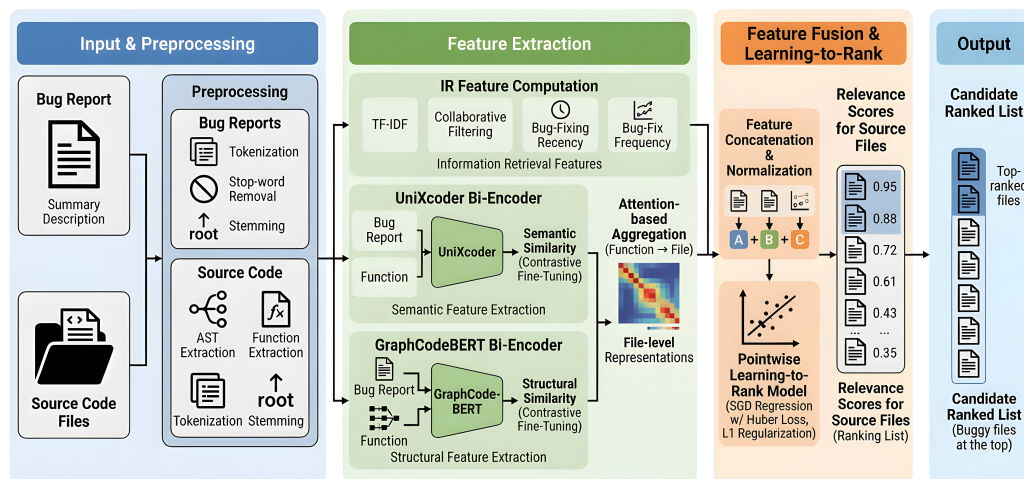


Figure 1. Overview of the DuaLoc framework.

4.1. Problem Formulation and Preprocessing

Let $\mathcal{S} = \{s_1, \dots, s_{N_s}\}$ denote the source files of a project and $\mathcal{R} = \{r_1, \dots, r_{N_r}\}$ the associated bug reports, where $N_s = |\mathcal{S}|$ and $N_r = |\mathcal{R}|$. Given a report $r \in \mathcal{R}$, the task is to rank the files in $\mathcal{S}$ so that those whose modification is required to resolve r appear at the top.

Each report is split into its summary and description fields and represented as $r = (r_{\text{sum}}, r_{\text{desc}})$. On the code side, each source file is parsed and decomposed into its constituent functions/methods. We denote a generic function by f and the full corpus of functions by $\mathcal{F}$. Every source file is therefore associated with the list of its constituent functions $s_i = [f_1^{(i)}, \dots, f_{k_i}^{(i)}]$, where k_i is the number of functions in s_i . We use the function as our unit because it is self-contained both syntactically and semantically, and it usually fits within the encoder's input window. This avoids the truncation problems of fixed-window chunking, which tends to cut off variable definitions and break apart control-flow constructs.

A common preprocessing pipeline is then applied to both sides. We strip out punctuation and numeric literals, filter English stop-words, and split compound identifiers along camelCase and underscore boundaries, keeping the original form together with its sub-tokens (for example, `FileInputStream` $\rightarrow$ `FileInputStream`, `file`, `input`, `stream`). Finally, we reduce the remaining tokens to their stems using the Porter algorithm [36]. On the code side, we apply the same procedure uniformly to the identifiers, reserved keywords, and in-code comments of every extracted function.

4.2. Semantic and Structural Feature Extraction

Surface lexical matching breaks down whenever the developer and the codebase do not share terminology, resulting in a lexical gap. To address this, DuaLoc relies on two pre-trained encoders: UniXcoder [15], used as the semantic encoder to bridge the lexical gap, and GraphCodeBERT [14], used as the structural encoder to capture data-flow dependencies that pure token-level matching cannot capture. We deploy both in a bi-encoder setup, passing the bug report and the candidate function through two separate forward passes that share the same encoder weights. Then we compare the resulting representations using cosine similarity. We prefer a bi-encoder over a cross-encoder because each function embedding can be computed once and then cached for reuse.

4.2.1. Encoder Inputs and Embedding Extraction

Each encoder, denoted generically by $E(\cdot)$, is operated in encoder-only mode and applied to any input x . That input is either a bug-report component, such as the summary or the description, or the body of a source-code function. The input is tokenized into a subword sequence prefixed by the special classification token [CLS] and processed through the transformer stack. We then extract the final hidden state at the [CLS] position and adopt it as the embedding of x . This gives us a fixed-length vector that condenses the input into a single dense representation, ready for similarity computation:

$$v = E(x)_{[\text{CLS}]}. \quad (1)$$

UniXcoder encodes subword sequences for both the input bug report and the source file. In contrast, GraphCodeBERT accepts natural-language input for the report and augments the code input with variable nodes $\{n_1, \dots, n_k\}$ extracted from the function's abstract syntax tree (AST), as well as a graph-guided attention mask derived from the data-flow graph. This mask limits attention to code comment token pairs, variable nodes connected by data-flow edges, and each variable node with the code token that defines it.

4.2.2. Contrastive Fine-Tuning

The pre-trained weights of UniXcoder and GraphCodeBERT are not optimized for retrieval. We therefore adapt each encoder with a contrastive objective applied at the function level. For every bug report r , each function f^+ belonging to a file modified to fix r forms a positive pair (r, f^+) , and negatives are drawn in-batch from the fixing functions of the other reports in the mini-batch. Although the ground truth is available only at the file level, we apply contrastive fine-tuning at the function granularity by treating functions from bug-fixing files as weak positive instances. This allows the encoders to learn from localized code units while preserving consistency with the file-level localization objective.

Training proceeds with in-batch negatives. Within a mini-batch $\mathcal{B} = \{(r_i, f_i^+)\}_{i=1}^N$, the fixing functions of all N reports together form the candidate set. For each report r_i , its positive is then contrasted against the $N - 1$ fixing functions belonging to the other reports, which serve as negatives and turn the objective into an N -way classification. Batches are sampled under the constraint that no two reports share a fixing file, ensuring that genuine positives are never assigned negative labels. Each encoder produces ℓ_2 -normalized [CLS] embeddings, so that cosine similarity reduces to an inner product. The InfoNCE loss [16] is then defined as:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(v_{r_i}, v_{f_i^+})/\tau_c)}{\sum_{j=1}^N \exp(\cos(v_{r_i}, v_{f_j})/\tau_c)}, \quad (2)$$

where v_{r_i} and v_{f_j} are the embeddings of report r_i and the fixing function of report j , and $\tau_c > 0$ is the softmax temperature. For each encoder, separate instances of $\mathcal{L}_{\text{InfoNCE}}$ are minimized independently. The semantic and structural views adapt without interference from mutual gradients. The resulting embedding space displays the alignment and uniformity properties typical of contrastive learning, pulling fixing functions toward their bug report while pushing unrelated functions apart.

4.2.3. Function-to-File Aggregation

Although matching is performed at the file level, the encoders operate on individual functions. Therefore, a file-level representation must be aggregated from its constituent

function embeddings. We adopt a report-conditioned attention mechanism that learns which functions to emphasize for each query.

Given a bug report embedding v_r and the function embeddings $\{v_{f_1}, \dots, v_{f_k}\}$ of a file s , where k is the number of functions in s , a small two-layer attention network assigns each function a relevance score conditioned on the report. The report embedding is concatenated with each function embedding and passed through a hidden layer with a tanh nonlinearity. The output of this layer is then projected onto a learnable scoring vector. The resulting scalar scores are normalized across the file's functions by a softmax to yield attention weights α_j . The file-level embedding v_s is then the corresponding weighted sum of the function embeddings:

$$v_s = \sum_{j=1}^k \alpha_j v_{f_j}, \quad (3)$$

where the parameters of the attention network, the projection matrix, the scoring vector, and the bias are learned jointly with the ranker (Section 4.4).

The same file produces different file-level representations across queries because the attention weights α_j , and hence v_s , depend on v_r . The aggregation is instantiated four times per (r, s) pair, once for each combination of encoder $\in \{\text{UniXcoder}, \text{GraphCodeBERT}\}$ and report component $\in \{\text{sum}, \text{desc}\}$. Every instance produces four deep-learning similarity scores: $\sigma_{\text{sum}}^{\text{UniX}}, \sigma_{\text{desc}}^{\text{UniX}}, \sigma_{\text{sum}}^{\text{Graph}},$ and $\sigma_{\text{desc}}^{\text{Graph}}$. Each score is computed as the cosine similarity between the aggregated file embedding for that (encoder, component) pair and the matching report component embedding.

4.3. Information Retrieval Features

We adapt four classical information-retrieval (IR) features from Ye et al. [17], identified in their study as strong predictors of fault-prone files, which are surface lexical similarity (ϕ_1), collaborative filtering (ϕ_2), bug-fixing recency (ϕ_3), and bug-fixing frequency (ϕ_4). Together, these features exploit complementary lexical, historical, and temporal evidence.

A bug report r is represented by the concatenation of its summary and description fields, while a source file s is represented both as a whole document and as the set of its constituent functions. For the collaborative filtering feature, the history before (r, s) is represented by the concatenation of the summaries of past reports only, following Ye et al. [17]. Term weights are computed using the normalized TF-IDF scheme:

$$\begin{aligned} w_{t,d} &= nf_{t,d} \times idf_t, \\ nf_{t,d} &= 0.5 + 0.5 \cdot \frac{tf_{t,d}}{\max_{t' \in d} tf_{t',d}}, \\ idf_t &= \log \frac{N}{df_t}, \end{aligned} \quad (4)$$

where $tf_{t,d}$ denotes the raw frequency of term t in document d , df_t denotes the number of documents containing term t , N denotes the total number of documents in the corpus, and idf_t denotes the inverse document frequency. The double normalization in $nf_{t,d}$ prevents long files from dominating the similarity scores through raw token counts alone. The similarity between two documents is then computed as the inner product of their ℓ_2 -normalized weight vectors, $\text{sim}(r, s) = \mathbf{r}^\top \mathbf{s} / (\|\mathbf{r}\| \|\mathbf{s}\|)$, where $\mathbf{r}$ and $\mathbf{s}$ denote the TF-IDF vectors of r and s , respectively.

In large source files, unrelated vocabulary can reduce cosine similarity because faults are typically localized to a single function. The surface lexical similarity ϕ_1 takes the maximum over the whole-file similarity and the per-function similarities:

$$\phi_1(r, s) = \max\left(\{\text{sim}(r, s)\} \cup \{\text{sim}(r, f) \mid f \in s\}\right). \quad (5)$$

The collaborative filtering feature ϕ_2 captures the empirical tendency of similar reports to map to overlapping sets of files. Let $\text{before}(r, s)$ be the set of reports whose fixes were touched s strictly before r was filed. Then

$$\phi_2(r, s) = \text{sim}(r, \text{before}(r, s)), \quad (6)$$

with the convention that $\phi_2 = 0$ when s has no prior fix history.

Lastly, the recency feature ϕ_3 and the frequency feature ϕ_4 capture a prior on which files are likely to be buggy, derived from the project's version history. Writing $\text{last}(r, s)$ for the filing month of the most recent report in $\text{before}(r, s)$, we define

$$\phi_3(r, s) = \frac{1}{r.\text{month} - \text{last}(r, s).\text{month} + 1}, \quad (7)$$

$$\phi_4(r, s) = |\text{before}(r, s)|. \quad (8)$$

We apply min-max normalization for IR features before feature fusion (Section 4.4).

4.4. Learning-to-Rank with Fused Features

For every candidate (r, s) pair, the deep similarity scores derived from the dual-encoder representations of Section 4.2 are concatenated with the classical information-retrieval features defined in Section 4.3 to form a single feature vector:

$$\Phi(r, s) = [\sigma_{\text{sum}}^{\text{UniX}}, \sigma_{\text{desc}}^{\text{UniX}}, \sigma_{\text{sum}}^{\text{Graph}}, \sigma_{\text{desc}}^{\text{Graph}}, \phi_1, \phi_2, \phi_3, \phi_4]. \quad (9)$$

The resulting feature vector is then passed to a linear scorer.

$$y(r, s) = \mathbf{w}^\top \Phi(r, s). \quad (10)$$

The scorer is trained jointly with the four attention networks of Section 4.2.3 under point-wise regression. We use a Huber loss with an ℓ_1 penalty on $\mathbf{w}$ and optimize it by stochastic gradient descent. The pre-trained encoders are frozen, so only $\mathbf{w}$ and the attention parameters receive gradients. In place of binary labels, we adopt the ameliorated continuous target of Fejzer et al. [6]: for a report r with a buggy file set $B(r)$ and for each feature ϕ_k (and analogously each σ_k), the largest value $\max_{s' \in B(r)} \phi_k(r, s')$ observed among the buggy files is added to that feature's value on every buggy file. This ensures that every buggy file surpasses the non-buggy ones while still allowing fine-grained comparison between two buggy files and between two irrelevant ones along each coordinate of $\Phi(r, s)$.

Bug localization is severely class-imbalanced; each report has only a handful of relevant files among several thousand irrelevant ones. We adopt the hard-negative mining scheme of Ye et al. [17], retaining for each report the 200 non-fixed files with the highest surface lexical similarity ϕ_1 . These share vocabulary with the report yet are not associated with the fix, and are precisely the cases the ranker must learn to discriminate.

5. Experimental Design

In this section, we present the research questions, datasets studied, baselines, evaluation metrics, and configurations used throughout our experiments.

5.1. Research Questions

To evaluate the effectiveness of DuaLoc, this study addresses the following research questions:

- **RQ1: How does DuaLoc perform relative to existing bug-localization baselines?** This research question evaluates the overall retrieval effectiveness of DuaLoc compared to state-of-the-art approaches across benchmark datasets.
- **RQ2: To what extent does each feature group contribute to DuaLoc’s performance?** This research question presents an ablation study across three feature groups: IR features, semantic features, and structural features, to quantify their individual and combined contributions to ranking effectiveness.
- **RQ3: How do DuaLoc’s principal architectural components influence its effectiveness?** This research question isolates the contributions of contrastive fine-tuning and bug-report-conditioned attention aggregation by comparing model variants in which each component is individually disabled.

5.2. Benchmark Datasets

We adopt the benchmark released by Ye et al. [17], which is widely used and contains a large number of bug reports and source files [37]. The dataset comprises bug reports, source code references, ground-truth buggy files, API documentation, and oracle bug-to-file mappings. The benchmark covers six open-source Java projects: Eclipse Platform UI, JDT, BIRT, SWT, Tomcat, and AspectJ. The projects differ substantially in scale, ranging from 593 bug reports in AspectJ to 6,495 in the Eclipse Platform UI. Their key statistics are summarized in Table 1.

Table 1. Benchmark datasets.

Project	Time Range	# Bug Reports	# Source Files	# Functions
Eclipse Platform UI	Oct. 2001–Jan. 2014	6495	3454	69,080
JDT	Oct. 2001–Jan. 2014	6274	8184	180,048
BIRT	Jun. 2005–Dec. 2013	4178	6841	143,661
SWT	Feb. 2002–Jan. 2014	4151	2056	133,640
Tomcat	Jul. 2002–Jan. 2014	1056	1552	38,800
AspectJ	Mar. 2002–Jan. 2014	593	4439	48,829

5.3. Baselines

We compare DuaLoc against four baselines spanning the principal families of bug-localization techniques, including both IR-based and deep-learning-based approaches.

- **BugLocator** [4]. A classical IR-based approach that ranks candidate source files using a vector space model based on their textual similarity to the bug report and refines the ranking from previously fixed, lexically similar bugs.
- **YBL** [17]. A learning-to-rank method proposed by Ye et al. combines multiple source-code features, including an enriched API hierarchy, collaborative filtering, and bug-fix recency and frequency, to produce a ranked list of suspicious files.
- **DreamLoc** [18]. A deep-learning-based approach that employs an attention mechanism to model the relevance between bug-report terms and code snippets.
- **FLIM** [12]. A deep-learning method that leverages CodeBERT to learn function-level semantic representations and aggregates them to the file level for localization.

5.4. Evaluation Metrics

We assess DuaLoc using three standard ranking metrics widely adopted in the bug-localization literature: Top-K, Mean Average Precision (MAP), and Mean Reciprocal Rank (MRR).

- **Top-K.** Top-K measures the proportion of bug reports for which at least one buggy file appears within the top K positions of the ranked list returned by DuaLoc (with K typically set to 1, 5, and 10). A bug report is regarded as successfully localized if any of its associated buggy files are contained in the top K results. Higher values indicate better performance, reflecting the practical observation that developers typically inspect only the first few recommended files.
- **Mean Average Precision (MAP).** MAP captures the quality of the ranking when a bug report is associated with multiple buggy files. For a given bug report with buggy-file set S , the average precision (AvgP) is defined as the mean of the precision values evaluated at the rank of each file in S :

$$\text{AvgP} = \sum_{n=1}^M \frac{P(n) \times \text{pos}(n)}{\text{number of buggy files}}, \quad (11)$$

where n is the rank of a source file, M is the number of ranked files, $P(n)$ is the precision at rank n , and $\text{pos}(n)$ indicates whether the n -th file is buggy. MAP is then the mean of AvgP over all bug reports in $\mathcal{R}$:

$$\text{MAP} = \frac{1}{|\mathcal{R}|} \sum_{i=1}^{|\mathcal{R}|} \text{AvgP}_i. \quad (12)$$

- **Mean Reciprocal Rank (MRR).** For an individual bug report, the reciprocal rank is the multiplicative inverse of the position at which the first buggy file appears in the returned list. MRR is then defined as the mean of the reciprocal ranks over the set of bug reports $\mathcal{R}$, and is computed as follows:

$$\text{MRR} = \frac{1}{|\mathcal{R}|} \sum_{i=1}^{|\mathcal{R}|} \frac{1}{\text{rank}_i}, \quad (13)$$

where rank_i is the position of the first buggy file in the returned list for the i -th bug report in $\mathcal{R}$.

5.5. Configuration Setup

All experiments were conducted on a single NVIDIA RTX 5090 GPU. UniXcoder and GraphCodeBERT are fine-tuned independently using the same recipe. This setup uses the AdamW optimizer, a learning rate of 2×10^{-5} , a weight decay of 0.01, and a batch size of 64. The contrastive temperature is set to $\tau_c = 0.07$, and each encoder is trained for five epochs.

For a fair comparison with prior work, we adopt the consecutive-fold evaluation protocol of Ye et al. [17], which has subsequently been employed by FLIM [12]. The bug reports of each project are ordered chronologically and partitioned into K equally sized folds. The model is trained on fold n and evaluated on fold $n + 1$ for every $1 \leq n < K$. This method maintains temporal validity by preventing information from future reports from informing the localization of earlier ones. The number of folds in each project is: two for AspectJ and Tomcat, eight for BIRT and SWT, and twelve for Eclipse Platform UI and JDT. For BugLocator [4] and DreamLoc [18], we use the results reported by FLIM [12], since

its authors had already re-run both baselines under the same partitioning we adopt. The numbers for YBL and FLIM are taken directly from their original papers [12,17].

6. Results and Discussion

This section presents our empirical findings, organized by research question. Section 6.1 (RQ1) compares DuaLoc with four baselines across all six projects in terms of Top-K, MAP, and MRR. Section 6.2 (RQ2) ablates the three feature groups to quantify the relative contribution of the IR, UniXcoder, and GraphCodeBERT signals. Section 6.3 (RQ3) isolates the effects of contrastive fine-tuning and bug-report-conditioned attention.

6.1. RQ1: Localization Performance

Table 2 reports the performance of DuaLoc under its full configuration, in which both UniXcoder and GraphCodeBERT are fine-tuned, function-level embeddings are aggregated, and the resulting neural relevance scores are fused with classical IR features within a learning-to-rank model. Across the six benchmark projects and five evaluation metrics, DuaLoc attains the best result in a large majority of (project, metric) combinations, establishing it as the strongest method overall. A small number of configurations remain in which the most competitive baseline, FLIM, retains a marginal advantage. Because developers routinely inspect only the highest-ranked candidates, the Top-K metrics have the greatest operational significance. Relative to FLIM, DuaLoc improves Top-1 accuracy on five of the six projects by 6.8%, 5.3%, 16.7%, 7.9%, and 4.0% on Eclipse Platform UI, JDT, BIRT, SWT, and Tomcat, respectively. It also delivers uniform gains on Top-5 and Top-10 across all six. Comparable improvements are observed for the rank-quality metrics, with MAP increasing by 8.3%, 7.3%, 9.5%, 6.7%, and 5.7% on Eclipse Platform UI, JDT, BIRT, SWT, and Tomcat, respectively. MRR is increasing by 5.5%, 11.1%, 7.7%, 3.3%, and 4.2% on Eclipse Platform UI, BIRT, SWT, Tomcat, and AspectJ, respectively.

Two projects depart from this overall pattern, and in both cases the departure is consistent with the underlying training conditions. On JDT, our model trails FLIM on MRR (0.48 versus 0.50) while leading on the remaining four metrics. On AspectJ, the smallest project, DuaLoc is narrowly outperformed on Top-1 (0.37 versus 0.38) and MAP (0.37 versus 0.39) yet has an advantage on Top-5, Top-10, and MRR. The AspectJ variant reflects the limited supervision available for contrastive fine-tuning in this smallest project and does not alter the overall conclusion.

The baseline comparisons indicate a coherent pattern when viewed collectively. DuaLoc substantially outperforms the IR-only techniques (BugLocator and YBL). That proves the documented insufficiency of term-frequency statistics for bridging the lexical gap between natural-language reports and programming-language identifiers. The margin over DreamLoc is likewise considerable, attributable largely to DreamLoc's fixed-length segmentation. DuaLoc's function-level decomposition preserves the integrity of each code unit, whereas DreamLoc fragments method bodies. Because FLIM is the baseline closest to DuaLoc in architecture and design, improvement relative to FLIM matters most. DuaLoc pairs a semantically oriented encoder (UniXcoder) with a structurally aware encoder (GraphCodeBERT), which FLIM's single CodeBERT encoder does not capture. It uses a bug-report-conditioned attention that adaptively concentrates weight on a single method for well-localized defects and distributes it across methods for diffuse issues, in contrast to FLIM's uniform pooling. We use the InfoNCE contrastive objective to directly shape the embedding geometry for retrieval, rather than relying on FLIM's cross-entropy classification objective.

Table 2. Comparison of DuaLoc with baselines on the benchmark projects.

Project	Method	Top-1	Top-5	Top-10	MAP	MRR
Eclipse Platform UI	BugLocator	0.27	0.49	0.60	0.31	0.37
	YBL	0.39	0.65	0.75	0.44	0.51
	DreamLoc	0.29	0.53	0.60	0.23	0.36
	FLIM	0.44	0.69	0.77	0.48	0.55
	DuaLoc	0.47	0.75	0.80	0.52	0.58
JDT	BugLocator	0.19	0.40	0.51	0.23	0.30
	YBL	0.35	0.63	0.74	0.39	0.47
	DreamLoc	0.33	0.59	0.70	0.38	0.46
	FLIM	0.38	0.65	0.76	0.41	0.50
	DuaLoc	0.40	0.68	0.78	0.44	0.48
BIRT	BugLocator	0.11	0.25	0.32	0.14	0.18
	YBL	0.14	0.29	0.38	0.16	0.21
	DreamLoc	0.17	0.35	0.44	0.19	0.26
	FLIM	0.18	0.36	0.45	0.21	0.27
	DuaLoc	0.21	0.39	0.48	0.23	0.30
SWT	BugLocator	0.19	0.38	0.51	0.25	0.28
	YBL	0.33	0.63	0.75	0.40	0.46
	DreamLoc	0.31	0.59	0.69	0.39	0.44
	FLIM	0.38	0.69	0.79	0.45	0.52
	DuaLoc	0.41	0.73	0.81	0.48	0.56
Tomcat	BugLocator	0.36	0.62	0.71	0.43	0.48
	YBL	0.43	0.71	0.80	0.49	0.55
	DreamLoc	0.40	0.72	0.81	0.48	0.54
	FLIM	0.50	0.73	0.81	0.53	0.60
	DuaLoc	0.52	0.76	0.85	0.56	0.62
AspectJ	BugLocator	0.20	0.48	0.57	0.22	0.32
	YBL	0.36	0.53	0.64	0.37	0.44
	DreamLoc	0.25	0.46	0.58	0.27	0.37
	FLIM	0.38	0.61	0.69	0.39	0.48
	DuaLoc	0.37	0.63	0.71	0.37	0.50

6.2. RQ2: Feature Ablation Study

This experiment examines the contribution of each of the three feature families: UniX-coder semantic similarity scores, GraphCodeBERT structural similarity scores, and classical IR features, to the final ranking produced by the complete pipeline. To this end, the learning-to-rank model was retrained from scratch on three reduced feature sets, each restricted to a single family, and compared against the complete configuration. We denote by DuaLoc_S the variant restricted to the scores produced by UniXcoder [15], by DuaLoc_G the variant restricted to the scores produced by GraphCodeBERT [14], by DuaLoc_{IR} the variant restricted to the IR features, and by DuaLoc (full model) the fusion of all three families. The corresponding results are reported in Table 3.

The dominant observation in Table 3 is that the full model achieves the highest value across all (project, metric) combinations. Averaged over the six projects, MAP grows from 0.35 for the most powerful single-family variant (DuaLoc_{IR}) to 0.43 for the full model, a relative improvement of approximately 25% (Figure 2). The improvement is roughly 44% over DuaLoc_S (mean MAP 0.30) and approximately 54% over DuaLoc_G (mean MAP 0.28). The same holds for the evaluation metrics averaged across projects; fusing the two encoders with the IR features increases Top-1 accuracy from 0.31 to 0.40 and MRR from 0.41 to 0.51. This design implies that the three feature families contribute complementary,

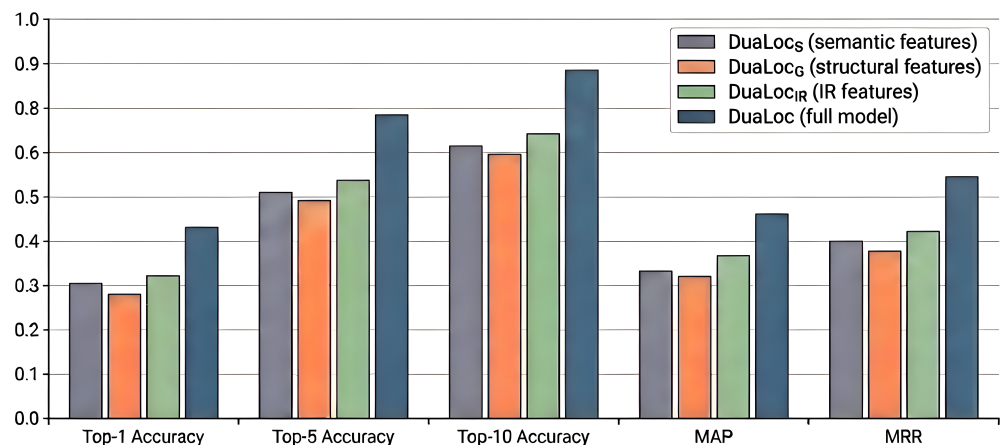


Figure 2. Contribution of each feature group across projects.

non-redundant information. The IR features recover the match when a bug report and a candidate file share substantial vocabulary. They fail to do so if the lexical overlap narrows. The two pre-trained encoders are complementary: UniXcoder projects natural language and source code into a shared semantic space, while GraphCodeBERT reaches the target file through its data-flow structure.

Considered in isolation, the IR features include the strongest individual family on every project, since three of their four members (collaborative filtering, recency, and frequency) function as historical priors rather than as purely lexical matchers. Their advantage is most pronounced on AspectJ, where DuaLoc_{IR} reaches a Top-1 of 0.33 against 0.17 for DuaLoc_S and 0.12 for DuaLoc_G. Since AspectJ is the smallest project and offers the contrastively fine-tuned encoders very little project-specific data to learn from. However, the IR family alone never matches the full configuration. This indicates that the historical signal must be paired with learned semantic and structural representations.

The comparison between the two deep-learning variants is more subtle, as their relative order is project-dependent rather than uniform across projects. DuaLoc_G surpasses DuaLoc_S on all five metrics for JDT (Top-1 0.29 vs. 0.27, MAP 0.33 vs. 0.31, MRR 0.39 vs. 0.37) and for SWT (Top-1 0.30 vs. 0.27, MAP 0.37 vs. 0.33, MRR 0.42 vs. 0.38). One plausible explanation is that these two codebases show a higher incidence of faults that propagate along chains of variable dependencies, exactly the pattern that GraphCodeBERT's data-flow-aware pre-training is designed to capture [14]. The ordering reverses on the remaining four projects, Eclipse Platform UI, BIRT, Tomcat, and AspectJ, where DuaLoc_S is the stronger variant on every metric (e.g., on Tomcat, MAP 0.44 vs. 0.37 and MRR 0.50 vs. 0.43). This indicates that the cross-modal alignment learned by UniXcoder [15] provides more discriminative information on these codebases. The project variability provides direct empirical justification for retaining both encoders in the final model.

6.3. RQ3: Effectiveness of Architectural Components

This experiment isolates the contributions of the two architectures on top of the dual-encoder base. It is the contrastive fine-tuning of the two encoders and the bug-report-conditioned attention that aggregates function embeddings into file-level representations. We retrain DuaLoc under three reduced configurations and compare them against the complete model. We denote by DuaLoc_{base} the pre-trained encoders used without con-

Table 3. Ablation study—different feature groups.

Variant	Eclipse UI	JDT	BIRT	SWT	Tomcat	AspectJ
<i>Top-1</i>						
DuaLoc _S	0.33	0.27	0.12	0.27	0.40	0.17
DuaLoc _G	0.29	0.29	0.10	0.30	0.33	0.12
DuaLoc _{IR}	0.34	0.31	0.13	0.31	0.42	0.33
DuaLoc	0.47	0.40	0.21	0.41	0.52	0.37
<i>Top-5</i>						
DuaLoc _S	0.57	0.50	0.27	0.52	0.62	0.34
DuaLoc _G	0.51	0.54	0.23	0.57	0.55	0.28
DuaLoc _{IR}	0.59	0.55	0.28	0.59	0.65	0.51
DuaLoc	0.75	0.68	0.39	0.73	0.76	0.63
<i>Top-10</i>						
DuaLoc _S	0.67	0.61	0.36	0.63	0.72	0.34
DuaLoc _G	0.62	0.65	0.31	0.69	0.66	0.29
DuaLoc _{IR}	0.68	0.66	0.37	0.70	0.74	0.61
DuaLoc	0.80	0.78	0.48	0.81	0.85	0.71
<i>MAP</i>						
DuaLoc _S	0.39	0.31	0.14	0.33	0.44	0.19
DuaLoc _G	0.34	0.33	0.12	0.37	0.37	0.15
DuaLoc _{IR}	0.41	0.34	0.15	0.38	0.46	0.34
DuaLoc	0.52	0.44	0.23	0.48	0.56	0.37
<i>MRR</i>						
DuaLoc _S	0.44	0.37	0.19	0.38	0.50	0.26
DuaLoc _G	0.39	0.39	0.16	0.42	0.43	0.21
DuaLoc _{IR}	0.45	0.41	0.20	0.44	0.52	0.43
DuaLoc	0.58	0.48	0.30	0.56	0.62	0.50

*DuaLoc_S uses only the semantic features, DuaLoc_G uses only the structural features, DuaLoc_{IR} uses only the IR features, and DuaLoc denotes the full model.

trastive adaptation, and function embeddings are combined by mean pooling. DuaLoc_{ft−} is the variant that removes contrastive fine-tuning while keeping the attention module. DuaLoc_{att−} is the variant that removes the attention module while keeping contrastive fine-tuning. The corresponding results are reported in Table 4 and visualized in Figure 3.

The full model attains the highest value in nearly every (project, metric) entry in Table 4, with only one exception: Top-1 on AspectJ. This is consistent with the RQ1 observation that AspectJ, the smallest project, provides too little project-specific supervision for contrastive fine-tuning. Removing both architectures' contributions in DuaLoc_{base} reduces top-1 accuracy from 0.40 to 0.33, MAP from 0.43 to 0.35, and MRR from 0.51 to 0.41, corresponding to relative decreases of roughly 18–20% averaged over the projects. The largest relative reduction occurs on BIRT, where Top-1 accuracy falls from 0.21 to 0.15. The large drop in BIRT shows that deep-learning mechanisms bear the greatest share of the localization effort, especially in how they are shaped by these two design choices. This occurs precisely in the project where the lexical gap between bug reports and the underlying source code is widest.

In the comparison between the two single-component ablations, DuaLoc_{ft−} attention without fine-tuning surpasses DuaLoc_{att−} fine-tuning without attention on Top-1 (0.38 versus 0.36), MAP (0.40 versus 0.39), and MRR (0.47 versus 0.45) averaged across the projects. Relative to DuaLoc_{base}, attention alone raises Top-1 by approximately 15% and fine-tuning alone by approximately 11%. As shown in Table 4, DuaLoc_{ft−} is in fact the strongest reduced variant on five of the six projects and across every metric. In BIRT, the

Table 4. Ablation study—different architectural components.

Variant	Eclipse UI	JDT	BIRT	SWT	Tomcat	AspectJ
<i>Top-1</i>						
DuaLoc _{base}	0.39	0.32	0.15	0.33	0.43	0.33
DuaLoc _{ft−}	0.44	0.39	0.17	0.40	0.50	0.38
DuaLoc _{att−}	0.42	0.36	0.19	0.36	0.48	0.35
DuaLoc	0.47	0.40	0.21	0.41	0.52	0.37
<i>Top-5</i>						
DuaLoc _{base}	0.59	0.55	0.29	0.58	0.61	0.51
DuaLoc _{ft−}	0.66	0.63	0.32	0.67	0.71	0.60
DuaLoc _{att−}	0.62	0.60	0.35	0.63	0.67	0.56
DuaLoc	0.75	0.68	0.39	0.73	0.76	0.63
<i>Top-10</i>						
DuaLoc _{base}	0.66	0.63	0.35	0.66	0.67	0.58
DuaLoc _{ft−}	0.77	0.73	0.40	0.76	0.78	0.67
DuaLoc _{att−}	0.71	0.70	0.43	0.72	0.74	0.63
DuaLoc	0.80	0.78	0.48	0.81	0.85	0.71
<i>MAP</i>						
DuaLoc _{base}	0.42	0.35	0.18	0.38	0.45	0.32
DuaLoc _{ft−}	0.47	0.41	0.20	0.45	0.53	0.36
DuaLoc _{att−}	0.45	0.39	0.22	0.42	0.49	0.35
DuaLoc	0.52	0.44	0.23	0.48	0.56	0.37
<i>MRR</i>						
DuaLoc _{base}	0.47	0.39	0.22	0.44	0.50	0.41
DuaLoc _{ft−}	0.54	0.47	0.25	0.53	0.59	0.47
DuaLoc _{att−}	0.51	0.46	0.27	0.48	0.55	0.45
DuaLoc	0.58	0.48	0.30	0.56	0.62	0.50

*DuaLoc_{base} denotes the backbone alone; DuaLoc_{ft−} removes contrastive fine-tuning; DuaLoc_{att−} removes the attention module; and DuaLoc denotes the full model.

order is reversed, with (DuaLoc_{att−}) leading across all five metrics because of BIRT's wide lexical gap. The complementarity is also evident in the full model, where combining both mechanisms yields better performance than using either alone.

7. Threats to Validity

In this section, we discuss the potential threats that may affect the validity of our study and the measures we took to mitigate them.

7.1. Construct Validity

Construct validity concerns whether the evaluation metrics we used genuinely reflect the effectiveness of bug localization techniques. To mitigate this threat, we rely on three well-established metrics, namely Top-K, MAP, and MRR, all of which have been extensively adopted in prior bug localization research [4,12,17,18,28]. Using these three metrics jointly reduces the risk that a technique performs well on one aspect of ranking while performing poorly on another. Top-K reflects the practical utility for developers who typically inspect only the top few suggestions. MAP captures overall ranking quality when a bug involves multiple buggy files, whereas MRR emphasizes how quickly the first buggy file is found.

7.2. External Validity

Threats to external validity concern the generalizability of our findings. The six subject projects in the Ye et al. benchmark are exclusively open-source Java systems, so

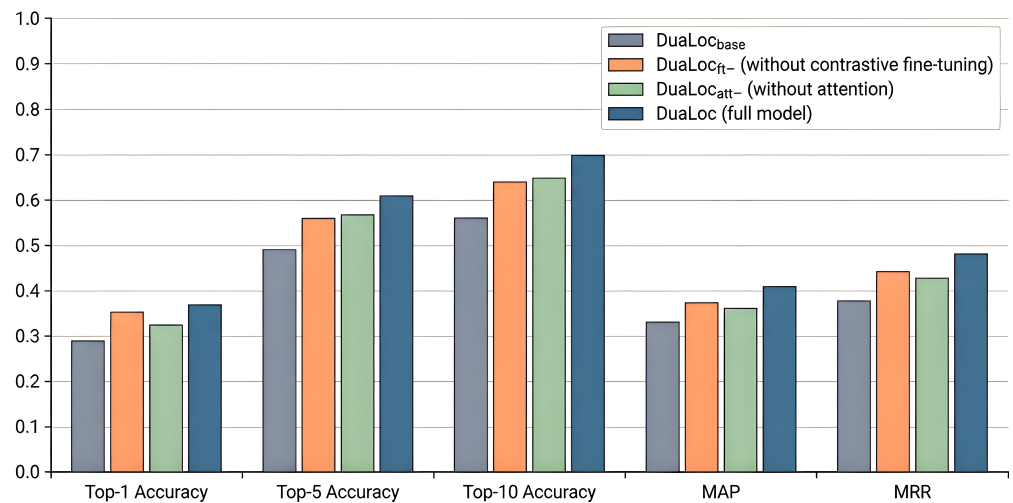


Figure 3. Effectiveness of architectural components across projects.

our conclusions may not generalize directly to other languages or proprietary industrial codebases. In such codebases, bug reports are often shorter, noisier, or more domain-specific. We partially mitigate this by selecting projects that span heterogeneous environments, including GUI frameworks (Eclipse Platform UI, SWT), IDE tooling (JDT), reporting and data-visualization platforms (BIRT), web servers (Tomcat), and aspect-oriented language extensions (AspectJ). Although this field diversity strengthens the generalizability of our findings to open-source Java systems, it does not eliminate the risk that effectiveness may differ in other programming languages or in industrial repositories. Our results should therefore be interpreted within this scope.

7.3. Internal Validity

The main threat to internal validity lies in potential errors in our feature collection and implementation of the technique. To reduce this threat, we built our pipeline on top of well-established tools, including PyTorch and the HuggingFace Transformers library for loading UniXcoder [15] and GraphCodeBERT [14], tree-sitter for AST-based function extraction, and NLTK for tokenization and stemming. For the IR side of the pipeline, we followed the feature definitions of Ye et al. [17] closely, ensuring that the classical features we use are computed in the same way as in the original work.

8. Conclusion

In this paper, we presented a bug localization framework built around a dual-encoder design that pairs UniXcoder for cross-modal semantics with GraphCodeBERT for data-flow structure. A bug-report-conditioned attention mechanism aggregates function embeddings into query-dependent file representations. The encoders are fine-tuned with a contrastive objective that directly shapes the embedding space for the ranking task. The resulting neural similarity scores are fused with classical IR features inside a learning-to-rank model. DualLoc outperformed four baselines from the IR, learning-to-rank, attention-based, and PLM-based families, achieving the best result on the vast majority of project metric combinations. Complementary studies showed both the value of combining heterogeneous evidence and the stability of the underlying design choices. Feature-group experiments showed that the IR, semantic, and structural signals each contribute information that the

others do not fully recover. We also ran ablation studies to show that both the contrastive objective and the bug-report-conditioned attention mechanism improve performance.

Several directions remain available. First, the evaluation should be extended to industrial codebases and other programming languages such as Python and C++. Second, additional structural views of the code, such as call graphs, could be incorporated to test whether they provide information that the two encoders miss while keeping inference cost manageable. Third, cross-project transfer and few-shot adaptation warrant closer study, as the AspectJ results suggest that contrastive fine-tuning still requires a nontrivial amount of project-specific supervision. Finally, we intend to construct a method-level annotated dataset and adapt the framework to operate at that granularity, giving developers a more precise recommendation than a file-level result.

Author Contributions: Conceptualization, A.A.; methodology, A.A.; software, A.A.; validation, A.A. and M.A.; formal analysis, A.A.; investigation, A.A.; resources, A.A.; data curation, A.A.; writing—original draft preparation, A.A.; writing—review and editing, A.A. and M.A.; visualization, A.A.; supervision, M.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The dataset used in this study is publicly available. It can be accessed through the figshare repository at <http://dx.doi.org/10.6084/m9.figshare.951967>.

Acknowledgments: During the preparation of this manuscript, the authors used Nano Banana Pro for the purpose of enhancing the figures and ChatGPT for language proofreading and editing. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Mohsen, A.M.; Hassan, H.; Moawad, R.; Makady, S. A review on software bug localization techniques using a motivational example. *Int. J. Adv. Comput. Sci. Appl.* **2022**, *13*, 251–262. <https://doi.org/10.14569/IJACSA.2022.0130231>
2. Zamfirov, F. A literature review on different types of empirically evaluated bug localization approaches. *arXiv* **2022**, arXiv:2212.11774. <https://doi.org/10.48550/arXiv.2212.11774>
3. Sangwan, O.P. Review of text mining techniques for software bug localization. In Proceedings of the 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence), Noida, India, January 2019; pp. 192–195. <https://doi.org/10.1109/CONFLUENCE.2019.8776959>
4. Zhou, J.; Zhang, H.; Lo, D. Where should bugs be fixed? More accurate information retrieval-based bug localization based on bug reports. In Proceedings of the 34th International Conference on Software Engineering (ICSE), Zurich, Switzerland, June 2012; pp. 14–24. <https://doi.org/10.1109/ICSE.2012.6227210>
5. Saha, R.K.; Lease, M.; Khurshid, S.; Perry, D. Improving bug localization using structured information retrieval. In Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), Silicon Valley, CA, USA, November 2013; pp. 345–355. <https://doi.org/10.1109/ASE.2013.6693093>
6. Fejzer, M.; Narebski, J.; Przymus, P.; Stencel, K. Tracking buggy files: New efficient adaptive bug localization algorithm. *IEEE Trans. Softw. Eng.* **2021**, *48*, 2557–2569. <https://doi.org/10.1109/TSE.2021.3064447>
7. Lam, A.N.; Nguyen, A.T.; Nguyen, H.A.; Nguyen, T.N. Combining deep learning with information retrieval to localize buggy files for bug reports. In Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), Lincoln, NE, USA, November 2015; pp. 476–481. <https://doi.org/10.1109/ASE.2015.73>
8. Huo, X.; Li, M.; Zhou, Z.-H. Learning unified features from natural and programming languages for locating buggy source code. In Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI), New York, NY, USA, July 2016; pp. 1606–1612.

9. Yang, G.; Lee, B. Utilizing topic-based similar commit information and CNN-LSTM algorithm for bug localization. *Symmetry* **2021**, *13*, 406. <https://doi.org/10.3390/sym13030406>
10. Huo, X.; Li, M.; Zhou, Z.-H. Control flow graph embedding based on multi-instance decomposition for bug localization. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, February 2020; Vol. 34, pp. 4223–4230. <https://doi.org/10.1609/aaai.v34i04.5844>
11. Xiao, Y.; Keung, J.; Bennin, K.E.; Mi, Q. Improving bug localization with word embedding and enhanced convolutional neural networks. *Inf. Softw. Technol.* **2019**, *105*, 17–29. <https://doi.org/10.1016/j.infsof.2018.08.002>
12. Liang, H.; Hang, D.; Li, X. Modeling function-level interactions for file-level bug localization. *Empir. Softw. Eng.* **2022**, *27*, 186. <https://doi.org/10.1007/s10664-022-10237-z>
13. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; Zhou, M. CodeBERT: A pre-trained model for programming and natural languages. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP, Online, November 2020; pp. 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
14. Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S.; Tufano, M. GraphCodeBERT: Pre-training code representations with data flow. *arXiv* **2020**, arXiv:2009.08366. <https://doi.org/10.48550/arXiv.2009.08366>
15. Guo, D.; Lu, S.; Duan, N.; Wang, Y.; Zhou, M.; Yin, J. UniXcoder: Unified cross-modal pre-training for code representation. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), Dublin, Ireland, May 2022; Vol. 1, pp. 7212–7225. <https://doi.org/10.18653/v1/2022.acl-long.499>
16. van den Oord, A.; Li, Y.; Vinyals, O. Representation learning with contrastive predictive coding. *arXiv* **2018**, arXiv:1807.03748. <https://doi.org/10.48550/arXiv.1807.03748>
17. Ye, X.; Bunescu, R.; Liu, C. Mapping bug reports to relevant files: A ranking model, a fine-grained benchmark, and feature evaluation. *IEEE Trans. Softw. Eng.* **2016**, *42*, 379–402. <https://doi.org/10.1109/TSE.2015.2479232>
18. Qi, B.; Sun, H.; Yuan, W.; Zhang, H.; Meng, X. DreamLoc: A deep relevance matching-based framework for bug localization. *IEEE Trans. Rel.* **2022**, *71*, 235–249. <https://doi.org/10.1109/TR.2021.3104728>
19. Kim, J.; Kim, J.; Lee, E. VFL: Variable-based fault localization. *Inf. Softw. Technol.* **2019**, *107*, 179–191. <https://doi.org/10.1016/j.infsof.2018.11.009>
20. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Minneapolis, MN, USA, June 2019; pp. 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
21. Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv* **2019**, arXiv:1907.11692. <https://doi.org/10.48550/arXiv.1907.11692>
22. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In Proceedings of Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, December 2017; pp. 5998–6008.
23. Xiao, X.; Pan, Y.; Zhang, B.; Hu, G.; Li, Q.; Lu, R. ALBFL: A novel neural ranking model for software fault localization via combining static and dynamic features. *Inf. Softw. Technol.* **2021**, *139*, 106653. <https://doi.org/10.1016/j.infsof.2021.106653>
24. Peng, Z.; Xiao, X.; Hu, G.; Sangaiah, A.K.; Atiquzzaman, M.; Xia, S. ABFL: An autoencoder based practical approach for software fault localization. *Inf. Sci.* **2020**, *510*, 108–121. <https://doi.org/10.1016/j.ins.2019.08.077>
25. Wang, S.; Lo, D. Amalgam+: Composing rich information sources for accurate bug localization. *J. Softw. Evol. Process* **2016**, *28*, 921–942. <https://doi.org/10.1002/smr.1801>
26. Wang, S.; Lo, D. Version history, similar report, and structure: Putting them together for improved bug localization. In Proceedings of the 22nd International Conference on Program Comprehension (ICPC), Hyderabad, India, June 2014; pp. 53–63. <https://doi.org/10.1145/2597008.2597148>

27. Youm, K.C.; Ahn, J.; Kim, J.; Lee, E. Bug localization based on code change histories and bug reports. In Proceedings of the 2015 Asia-Pacific Software Engineering Conference (APSEC), New Delhi, India, December 2015; pp. 190–197. <https://doi.org/10.1109/APSEC.2015.23>
28. Wen, J.; Wu, R.; Cheung, S.C. Locus: Locating bugs from software changes. In Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE), Singapore, September 2016; pp. 262–273. <https://doi.org/10.1145/2970276.2970359>
29. Gharibi, R.; Rasekh, A.H.; Sadreddini, M.H.; Fakhrahmad, S.M. Leveraging textual properties of bug reports to localize relevant source files. *Inf. Process. Manage.* **2018**, *54*, 1058–1076. <https://doi.org/10.1016/j.ipm.2018.07.004>
30. Ye, X.; Bunescu, R.; Liu, C. Learning to rank relevant files for bug reports using domain knowledge. In Proceedings of the 22nd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE), Hong Kong, China, November 2014; pp. 689–699. <https://doi.org/10.1145/2635868.2635874>
31. Xiao, Y.; Keung, J.; Mi, Q.; Bennin, K.E. Bug localization with semantic and structural features using convolutional neural network and cascade forest. In Proceedings of the 10th Asia-Pacific Symposium on Internetwork, Beijing, China, September 2018; pp. 101–111. <https://doi.org/10.1145/3210459.3210469>
32. Liang, H.; Sun, L.; Wang, M.; Yang, Y. Deep learning with customized abstract syntax tree for bug localization. *IEEE Access* **2019**, *7*, 116309–116320. <https://doi.org/10.1109/ACCESS.2019.2936948>
33. Asad, M.; Yasir, R.M.; Malek, S. Leveraging large language model for information retrieval-based bug localization. *arXiv* **2025**, arXiv:2508.00253.
34. Shao, S.; Yu, T. Enhancing IR-based fault localization using large language models. *arXiv* **2024**, arXiv:2412.03754. <https://doi.org/10.48550/arXiv.2412.03754>
35. Chakraborty, P.; Alfadel, M.; Nagappan, M. BLAZE: Cross-language and cross-project bug localization via dynamic chunking and hard example learning. *IEEE Trans. Softw. Eng.* **2025**, in press. <https://doi.org/10.1109/TSE.2025.3579574>
36. Porter, M.F. An algorithm for suffix stripping. *Program* **1980**, *14*, 130–137. <https://doi.org/10.1108/eb04681>
37. Kim, M.; Lee, E. Are datasets for information retrieval-based bug localization techniques trustworthy? *Empir. Softw. Eng.* **2021**, *26*, 35. <https://doi.org/10.1007/s10664-021-09946-8>

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.